FINANCIAL FRAUD DETECTION

Executive Summary — Data Cleaning, EDA & Model Evaluation

PMP Solutions | Standard Bank Hackathon — Data Science Track

1. Dataset Overview

636,262	11	849	0.133%
Transactions	Original Columns	Fraud Cases	Fraud Rate

Synthetic_Financial_Datasets_For_Fraud_Detection_resample.csv — modelled on the PaySim mobile-money simulation. Columns: step, type, amount, nameOrig, oldbalanceOrg, newbalanceOrig, nameDest, oldbalanceDest, newbalanceDest, isFraud (target), isFlaggedFraud.

Fraud detection here means finding roughly 1 transaction in every 750 — a severe class-imbalance problem that shapes every downstream decision: how the data is split, how models are evaluated, and why plain accuracy is meaningless as a success metric.

2. Data Cleaning

No missing values

Confirmed by df.isnull().sum() across all 11 columns — a genuinely clean input.

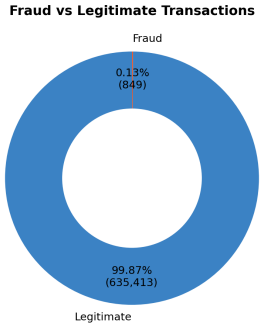
No exact duplicate rows — but the check is weak

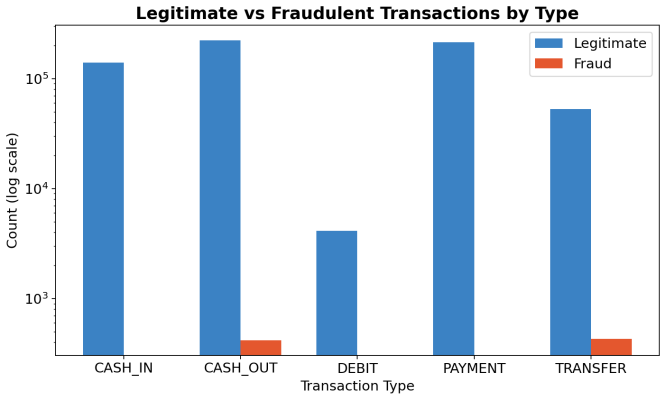
nameOrig/nameDest are near-unique account IDs, so an exact-row duplicate check almost can't fail. A stricter check (dropping ID columns, comparing step/type/amount/balances) is the more meaningful test.

Zero balances are not missing data

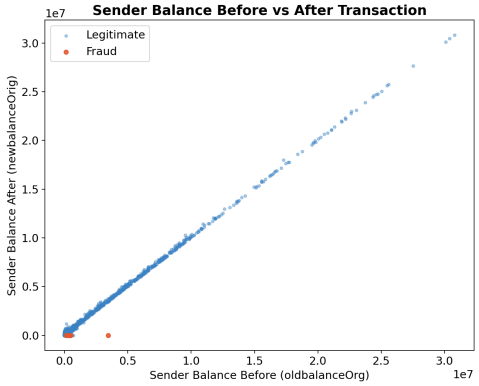
Merchant destination accounts ("M...") never have tracked balances in this dataset by design — 0 is a business rule, not a defect, and should not be imputed or dropped.

3. Key EDA Findings





Fraud is not spread evenly across transaction types — it occurs almost exclusively in CASH_OUT (417 cases) and TRANSFER (432 cases); CASH_IN, DEBIT, and PAYMENT show effectively zero fraud.



The strongest signal in the data: plotting sender balance before vs. after a transaction shows legitimate transactions spread naturally along the diagonal, while fraud cases cluster on the x-axis — the sender's account is almost always drained to exactly zero. This single behaviour ("the account gets fully emptied") underlies the most important engineered features.

4. Feature Engineering

Nine features were engineered from the raw balances to make fraud behaviour explicit to the models:

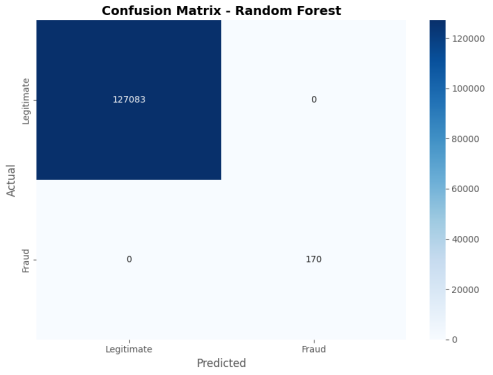
Feature	What it captures
BalanceChangeOrig / Dest	How much the sender's / receiver's balance actually moved.
AccountEmptied	Flags when the sender's balance hits exactly zero after the transaction.
AmountToBalanceRatio	Is the transaction close to the sender's entire balance? Fraud often ≈ 1.0.
RiskScore (0–100)	A hand-built composite of amount size, transaction type, and balance-drain ratio.
is_cash_out / is_transfer / …	One-hot flags — since fraud is concentrated in just two types.

Leakage removed before modelling

isFlaggedFraud was dropped — it's PaySim's own rule-based flag, not a raw attribute; keeping it would mean partly learning from another detector's decision. **nameOrig / nameDest** were dropped too — hundreds of thousands of unique IDs risk the model memorising specific accounts instead of learning generalisable behaviour. Train/test split: 80/20, stratified on isFraud, random_state=42.

5. Model Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Decision Tree	0.9999	0.9497	1.0000	0.9742	1.0000
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	0.9997	0.8057	1.0000	0.8924	1.0000



	Pred: Legit	Pred: Fraud
Actual: Legit	127,083	0
Actual: Fraud	0	170

Confusion matrix, Random Forest, on the held-out test set (127,253 transactions): 0 false alarms, 0 missed fraud cases. Combined with the perfect metrics above, this is the clearest signal that something in the feature set is separating the classes almost by construction — see Section 6.

6. Critical Finding: Too Good To Be True?

A perfect 1.0000 score across every metric on a real-world fraud task is a red flag, not a win. Three compounding reasons explain it here:

The synthetic data has a "tell"

Fraudulent CASH_OUT / TRANSFER transactions in this dataset almost always drain the sender's account completely (amount ≈ oldbalanceOrg, newbalanceOrig = 0). Real-world fraud is rarely this clean.

Engineered features encode that tell directly

AmountToBalanceRatio ≈ 1.0 and AccountEmptied = 1 are near-perfect proxies for this synthetic rule — the model isn't learning subtle behaviour, it's reading an artifact of how the data was generated.

RiskScore partially hard-codes domain rules

It's built from transaction type (TRANSFER/CASH_OUT weighted highest — matching exactly where fraud can occur) and balance-drain ratios — a composite that already "knows" the answer pattern before the model sees it.

7. Recommendations & Next Steps

1. **Re-test without the composite features** — drop RiskScore and re-check whether AmountToBalanceRatio / AccountEmptied alone still separate classes this cleanly, to isolate genuine signal from hard-coded rules.

2. **Report PR-AUC alongside ROC-AUC** — with 0.13% fraud, ROC-AUC is easy to inflate; precision-recall curves and the confusion matrix tell the real story of false alarms vs. missed fraud.

3. **Validate on held-out real-world data if possible** — PaySim is a strong training ground, but a synthetic generation rule is not the same as real fraud behaviour.

4. **Tune the decision threshold, not just the model** — in production, the cost of a false alarm vs. a missed fraud should set the classification cutoff, not the default 0.5.